DATA ENGINEERING & CLOUD · LESSON 6 OF 15

Apache Airflow: DAGs, Tasks, Scheduling and Retries

Orchestrate a data pipeline with Apache Airflow. Define tasks and dependencies as a directed acyclic graph (DAG), schedule runs with idempotent logic, configure retries and alerts, and learn the operational habits that keep a DAG reliable.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

By the end of this lesson you can

- Explain a DAG, its tasks and dependencies
- Write a DAG with a schedule, retries and failure alerts
- Keep tasks idempotent so that reruns and backfills are safe
- Diagnose a failed run from the task logs and the run history

What Airflow does

Apache Airflow schedules and monitors workflows. It does not process the data itself: it runs the tasks you define, in the right order, at the right time, and it records what happened. A task might call a SQL script, start a Spark job, or copy a file. Airflow decides when, in what order and what to do when a step fails.

CASE STUDY **The nightly close**

Every night a pipeline must load the day's files, run quality checks, rebuild the reporting tables and then refresh the dashboard. If the quality check fails, the dashboard must not refresh. If the load fails at 02:00, the team must know by 02:05, and the rerun must not duplicate data. An orchestrator makes each of those rules explicit.

DAGs: tasks with dependencies

A **DAG** (directed acyclic graph) is a set of tasks with arrows between them that show what must finish first. "Directed" means the arrows have a direction; "acyclic" means no task can depend on itself. The DAG is the pipeline's blueprint.

```
load_files ──► quality_check ──► build_gold ──► refresh_dashboard
                    │
                    └── (on failure) alert_owner
```

The following DAG is written for Airflow 2.x. It is **shown as code and not run**, because it needs an Airflow installation and a scheduler. Check the current Airflow documentation for the exact import paths in your version.

Python

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator

default_args = {
    "owner": "data-engineering",
    "retries": 3,                          # retry transient failures
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": True,
    "email": ["data-oncall@example.com"],
}

with DAG(
    dag_id="nightly_sales_close",
    start_date=datetime(2025, 1, 1),
    schedule="0 2 * * *",                  # every day at 02:00
    catchup=False,                         # do not run every missed day on deploy
    default_args=default_args,
    tags=["sales", "nightly"],
) as dag:
    load_files = BashOperator(task_id="load_files", bash_command="python load.py --date {{
ds }}")
    quality = PythonOperator(task_id="quality_check", python_callable=lambda: print("checks
passed"))
    build_gold = BashOperator(task_id="build_gold", bash_command="python build_gold.py --
date {{ ds }}")
    refresh = BashOperator(task_id="refresh_dashboard", bash_command="python refresh.py")

    load_files >> quality >> build_gold >> refresh
```

Read the key lines:

Line	Meaning
`schedule="0 2 * * *"`	A cron schedule: 02:00 every day
`retries: 3`	Retry each failing task up to three times
`retry_delay`	Wait five minutes between tries
`{{ ds }}`	Airflow's logical date, so each run processes its own day
`catchup=False`	Do not replay every past day when the DAG is first deployed
`>>`	"Must finish before": the dependency arrows

Idempotent tasks

Retries and backfills rerun tasks. A task that appends rows doubles them on every retry. Write each task so that running it for a given date twice gives the same result: overwrite the date's partition, or merge on a key. The `{{ ds }}` date in the command is what makes that possible, because the task always works on the same partition.

::::warn A task must know its date A task that uses `datetime.now()` instead of the run's logical date gives different results when it is rerun or backfilled. Pass the date in, and let Airflow supply it. ::::

Failure handling

Airflow gives several levels of response, which you should choose deliberately:

Mechanism	Use for
Retries with delay	Temporary network or database problems
Failure callback or email	Notifying the owner of a failed task
Trigger rules	Running a cleanup or alert task even when upstream fails
Sensors	Waiting for a file or a partner's data to arrive, with a timeout
SLA alerts	Warning when a task runs later than expected

A failure that should stop downstream work (a failed quality check) must block the tasks after it. A cleanup or alert task that must run regardless uses a trigger rule such as `all_done`.

Backfills

A **backfill** runs the DAG for past dates, usually after a bug fix. It is safe only if the tasks are idempotent. Run backfills in small date ranges, watch the load on the source and the warehouse, and check reconciliation after each batch.

Operating a DAG

- **Read the logs first.** The failed task's log shows the real error; the DAG graph shows where the failure stopped the chain.
- **Clear and rerun one task** after a fix, rather than rerunning the whole DAG, when the earlier tasks are already correct.
- **Keep DAGs small and readable.** A DAG with two hundred tasks is hard to debug; split it by data domain.
- **Put secrets in a connection store or environment variables,** never in the DAG file.
- **Test the DAG file** with the Airflow import check and a dry run before deploying.

INTERVIEW CORNER

How do you make an Airflow pipeline safe to rerun?

Make every task idempotent by working on the run's logical date and overwriting or merging its partition. Set bounded retries for transient errors, stop downstream tasks when a quality check fails, use `catchup=False` for new DAGs, and backfill in small batches with reconciliation after each one.

::::key What to remember - Airflow orchestrates tasks; it does not replace the tools that process the data. - A DAG is a set of tasks with dependencies; the schedule and retries are set per DAG or task. - Tasks must be idempotent and must use the run's date, not the current time. - Stop downstream work on failed checks; use trigger rules for cleanup and alerts. - Diagnose from the task log, fix, then clear and rerun the failed task. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does "acyclic" mean in a DAG?

A Tasks run at random

B No task can depend, directly or indirectly, on itself

C The DAG has no schedule

D Every task runs in parallel

Answer: B. A cycle would make the order of tasks impossible to decide.

2. Why should a task use the run's logical date rather than the current time?

A The current time is always wrong

B So that a rerun or backfill works on the same partition and gives the same result

C Airflow forbids the current time

D The current time is slower

Answer: B. Using the logical date makes reruns deterministic.

3. A quality check fails. Which task behaviour is correct?

A Let the dashboard refresh anyway

B Stop the downstream tasks so that the dashboard is not refreshed with bad data

C Retry the dashboard refresh forever

D Delete the quality check

Answer: B. Downstream work must depend on passing checks.

4. What does `catchup=False` prevent?

A Retries

B Failure alerts

C Running every missed past date when a new DAG is first deployed

D Scheduling

Answer: C. It stops an unexpected flood of historical runs.

5. Which mechanism waits for a file to arrive before continuing?

A A retry

B A trigger rule

C A sensor

D A DAG tag

Answer: C. Sensors wait for a condition, with a timeout.

EXERCISE **Design a DAG**

Describe, as a diagram and a list of tasks, a DAG that loads a daily sales file, checks that the row count matches the file's own footer, builds a monthly summary, and emails the owner only when a check fails. Write the dependency line, the schedule and the retry settings, and name the one task that must be idempotent and how you would make it so.

Show solution (worked answer)

Expected: five tasks, a sensor or a file check at the start, the quality check blocking the summary, a trigger rule or failure callback for the email, a daily cron schedule, retries for transient errors, and an idempotent summary that overwrites the month's partition.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.